# Python File Handling

Text files • CSV operations • Binary files & image handling

Mirai

Ashirwad Mishra

**Unit V | First-Year Programming**

Concepts, code examples, outputs, and revision questions

# Learning outcomes

By the end of this lesson, students should be able to…

- Explain why files are used for persistent storage.
- Read, write, append, and safely close text files.
- Choose file modes and handle common errors.
- Read and write CSV rows using Python’s csv module.
- Explain binary mode and work with bytes.
- Copy image files and use Pillow for basic image processing.

# Lesson roadmap

The presentation follows this order.

### 01 File handling

File concepts and modes
Reading and writing
Append, pointers, errors

### 02 CSV operations

CSV structure
reader / writer
DictReader / DictWriter

### 03 Binary & images

Bytes and binary modes
Copying files
Pillow: inspect, resize, grayscale

# 1. What is file handling?

Working with data stored in files.

### Why use files?

- Variables usually exist in memory while a program runs.
- Files can preserve data after the program ends.
- Useful for records, reports, settings, and logs.

### Common operations

Create • Open • Read • Write • Append • Close

Example: save student names and marks so they can be viewed later.

# Types of files

A useful first distinction is text versus binary.

### Text files

Store characters that can usually be read in a text editor.

Examples: .txt, .csv, .py, .html
Typical modes: r, w, a

### Binary files

Store bytes interpreted according to a file format.

Examples: .jpg, .png, .pdf, .mp3, .mp4
Typical modes: rb, wb

# Opening a file: open()

open() returns a file object.

```
file = open("sample.txt", "r")
data = file.read()
print(data)
file.close()
```

**Parts of open()**

- File name or path
- Mode
- Returned file object

**If the file does not exist, opening it in read mode normally raises FileNotFoundError.**

# File modes at a glance

Choose the mode based on the required operation.

| Mode | Purpose | Important note |
|---|---|---|
| **r** | Read | Must exist |
| **w** | Write; replace old content | Creates if missing |
| **a** | Append at the end | Creates if missing |
| **x** | Create new file exclusively | Error if already exists |
| **r+** | Read and write | Must exist |
| **b** | Binary modifier | e.g. rb, wb |
| **t** | Text modifier (default) | e.g. rt, wt |

**Exam point: "w" truncates existing content; "a" preserves it and adds at the end.**

# Reading text files

Three methods return file content in different forms.

### read()

Reads all remaining content as one string.

Useful for small files.

### readline()

Reads one line at a time.

Useful for line-by-line processing.

### readlines()

Returns a list of remaining lines.

May use more memory for large files.

**A line often includes the newline character \n at its end.**

# Example: read a file

Assume sample.txt contains three lines.

```
with open("sample.txt", "r", encoding="utf-8") as file:
 data = file.read()
 print(data)
```

**Why specify encoding?**

UTF-8 is a common text encoding. Specifying it makes text handling more predictable across systems.

**Tip: for large files, use "for line in file:" instead of loading all contents at once.**

# Writing to a text file

write() does not automatically add a newline.

```
with open("student.txt", "w", encoding="utf-8") as file:
file.write("Name: Rahul\n")
file.write("Marks: 85\n")
```

**File content**

Name: Rahul
Marks: 85

Opening with w replaces old content.

# Appending to a file

Use a mode that preserves existing contents.

```
with open("student.txt", "a", encoding="utf-8") as file:
file.write("Grade: A\n")
```

**After appending**

Name: Rahul
Marks: 85
Grade: A

New text is added at the end.

# Why use with open()?

The file is automatically closed when the block ends.

```
with open("sample.txt", "r", encoding="utf-8") as file:
    data = file.read()
    print(data)
# File closes automatically here
```

## Benefits

- Cleaner code
- Fewer resource leaks
- Closes the file even if an exception occurs inside the block

**Recommended practice: prefer with open() for ordinary file operations.**

# File pointer and error handling

tell() reports position; seek() changes it.

```
with open("sample.txt", "r", encoding="utf-8") as file:
print(file.tell())
print(file.read(5))
file.seek(0)
print(file.read(5))
```

```
try:
    with open("missing.txt", "r") as file:
        print(file.read())
except FileNotFoundError:
    print("File not found.")
```

Common errors include FileNotFoundError and PermissionError. Use try/except when the program should recover gracefully.

# Working with structured tabular data

Read and write rows using Python’s built-in csv module.

# What is a CSV file?

CSV stands for Comma-Separated Values.

```
Name,Age,Marks
Rahul,18,85
Priya,19,92
Aman,18,78
```

## Key ideas

- Each line usually represents a row.
- Commas separate fields.
- First row often contains column names.
- Commonly used for spreadsheet data.

**CSV is plain text with a convention for organizing rows and columns.**

# Writing CSV data with csv.writer()

writerow() writes one row; writerows() can write multiple rows.

```
import csv

with open("students.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(["Name", "Age", "Marks"])
    writer.writerow(["Rahul", 18, 85])
    writer.writerow(["Priya", 19, 92])
```

## Important details

- csv is built into Python.
- newline="" helps avoid unwanted blank rows on some platforms.
- Values are written as CSV fields.

# Reading CSV data with csv.reader()

Each row is returned as a list of strings.

```
import csv

with open("students.csv", "r", newline="") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

## Example output

['Name', 'Age', 'Marks']
['Rahul', '18', '85']
['Priya', '19', '92']

Convert values to int() or float() for calculations.

# CSV with dictionaries

DictWriter and DictReader work with column names.

```
import csv

with open("students.csv", "w", newline="") as file:
    fields = ["Name", "Marks"]
    writer = csv.DictWriter(file, fieldnames=fields)
    writer.writeheader()
    writer.writerow({"Name": "Rahul", "Marks": 85})
    writer.writerow({"Name": "Priya", "Marks": 92})
```

## Why use it?

- Keys correspond to column names.
- writeheader() writes the header row.
- Useful when data is represented as dictionaries.

# Reading CSV with DictReader

Each row can be accessed by column name.

```
import csv

with open("students.csv", "r", newline="") as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(row["Name"], row["Marks"])
```

**Output**

Rahul 85
Priya 92

CSV values are strings unless converted.

**Use reader() for list-based rows; use DictReader() when column names make code clearer.**

# CSV: common mistakes

Small details make CSV programs more reliable.

- Forgetting import csv.
- Forgetting newline="" when opening a CSV for writing.
- Assuming numeric fields are automatically converted.
- Using the wrong filename or working directory.
- Splitting rows manually on commas; quoted fields can contain commas.

# Working with bytes and images

Understand binary mode, copy raw files, and use Pillow for image operations.

# What is binary file handling?

Binary mode reads or writes bytes instead of decoded text.

### Text mode

Example: "r" or "w"

Reads or writes str values. Python handles text encoding and newline translation.

### Binary mode

Example: "rb" or "wb"

Reads or writes bytes. Used for raw data such as images, audio, video, and PDFs.

**A bytes object often displays with a leading b, for example: b"ABCD".**

# Binary file: a simple example

Byte values 65–68 represent ABCD in ASCII.

```
data = bytes([65, 66, 67, 68])

with open("data.bin", "wb") as file:
    file.write(data)

with open("data.bin", "rb") as file:
    result = file.read()

print(result)
```

**Output**

b"ABCD"

The returned object is bytes, not str—even if the bytes represent readable letters.

# Copying a binary file

This pattern can copy an image without interpreting its contents.

```
with open("photo.jpg", "rb") as source:
with open("photo_copy.jpg", "wb") as destination:
    while True:
        chunk = source.read(4096)
        if not chunk:
            break
        destination.write(chunk)
```

## Why use chunks?

- Reads 4096 bytes at a time.
- Avoids loading a large file fully into memory.
- Writes the same bytes to the destination.

# Image handling with Pillow

Use a library when you need to inspect or modify an image.

```
python -m pip install Pillow
```

**Pillow**

An external Python library for opening, displaying, resizing, converting, and saving images.

**The image must be in the working directory or you must provide its correct path.**

# Opening an image and inspecting it

Example output is illustrative; values depend on the image.

```
from PIL import Image

with Image.open("photo.jpg") as img:
    print("Format:", img.format)
    print("Size:", img.size)
    print("Mode:", img.mode)
    img.show()
```

**Example output**

Format: JPEG
Size: (800, 600)
Mode: RGB

size is (width, height) in pixels.

# Resizing an image

resize() creates a resized image; save() writes it to a file.

```
from PIL import Image

with Image.open("photo.jpg") as img:
    resized = img.resize((400, 300))
    resized.save("resized_photo.jpg")
```

**Remember**

- Size is (width, height).
- Creates a new output file.
- Original is not overwritten.

# Converting an image to grayscale

Pillow can convert an image to a different pixel mode.

```
from PIL import Image

with Image.open("photo.jpg") as img:
    gray = img.convert("L")
    gray.save("gray_photo.png")
```

### What does L mean?

L represents an 8-bit grayscale image. Each pixel stores a shade from black to white.

# Binary copying vs image processing

Choose the tool based on the task.

| Task | Suitable approach |
| --- | --- |
| **Copy image without changes** | open() with "rb" and "wb" |
| **Read raw image bytes** | open() with "rb" |
| **Display image** | Pillow: Image.open() and show() |
| **Resize image** | Pillow: resize() |
| **Convert to grayscale** | Pillow: convert("L") |
| **Save processed image** | Pillow: save() |

**Binary mode preserves bytes; Pillow understands image structure and provides image operations.**

# Common mistakes to avoid

Use this as a final checklist.

- Using "w" when old contents must be preserved; use "a" to append.
- Forgetting that write() does not automatically add a newline.
- Assuming CSV values are automatically numeric.
- Reading an image as ordinary text instead of binary data.
- Expecting open(..., "rb") to display or resize an image; use Pillow.
- Forgetting to install Pillow before importing PIL.

# Quick revision quiz

Try answering before checking your notes.

- 1. What is the difference between "w" and "a" modes?
- 2. Which method reads one line at a time?
- 3. Why use newline="" when writing CSV files?
- 4. What is the difference between str and bytes?
- 5. Which modes read and write raw image bytes?
- 6. Which Pillow method resizes an image?
- 7. What does img.size return?

# Quiz answer key

Use these to check understanding.

- 1. "w" replaces existing contents; "a" appends.
- 2. readline().
- 3. It helps prevent unwanted blank rows on some platforms.
- 4. str represents text; bytes represents raw byte values.
- 5. "rb" for reading and "wb" for writing.
- 6. resize().
- 7. A tuple: (width, height), measured in pixels.

# Take-home summary

Three sections, one consistent idea: choose the right tool for the data.

### File handling

Use open() modes carefully.
Read, write, append.
Prefer with open().
Handle errors.

### CSV operations

Use csv.reader / writer.
Use DictReader / DictWriter for named columns.
Convert strings before numeric calculations.

### Binary & images

Use rb / wb for raw bytes.
Copy files in chunks.
Use Pillow to inspect and process images.